Projet final — Construction d'un Knowledge Graph

Creatine & Supplements alimentaires

A4 – DIA4 – Marya LATIF, Faty LO – 2025/2026

Contents

1 Acquisition des donnees et extraction d'information **2**
1.1 Domaine et sources . . . 2
1.2 Pipeline de crawling . . . 2
1.3 Extraction d'entités nommées (NER) . . . 2
1.3.1 Trois cas d'ambiguite . . . 2

2 Construction de la KB, alignement et expansion **3**
2.1 Graphe initial (KB privée) . . . 3
2.2 Alignement d'entites avec Wikidata . . . 3
2.3 Alignement des predicats . . . 3
2.4 Expansion de la KB via SPARQL . . . 3

3 Raisonnement SWRL **4**
3.1 Règle sur family.owl . . . 4
3.2 Règle sur la KB creatine . . . 4

4 Knowledge Graph Embeddings (KGE) **4**
4.1 Préparation des données . . . 4
4.2 Modèles et résultats . . . 5
4.3 Sensibilité a la taille du KB . . . 5
4.4 Analyse des embeddings . . . 5

5 RAG sur RDF/SPARQL **6**
5.1 Setup . . . 6
5.2 Schema summary . . . 6
5.3 Pipeline NL–SPARQL avec self-repair . . . 6
5.4 Evaluation : baseline vs RAG . . . 6

6 Réflexion critique **7**

1. Acquisition des donnees et extraction d'information

1.1. Domaine et sources

On a choisi de travailler sur la **creatine et les supplements alimentaires**. C'est un domaine avec beaucoup de publications recentes en francais, ce qui facilitait le crawling, et suffisamment dense en entites nommees (molecules, chercheurs, institutions, dates d'etudes) pour construire un graphe interessant.

Cinq URLs de depart ont ete selectionnees pour couvrir differents angles :

1. *Protegez-Vous* (Canada) — benefices et usages de la creatine : https://www.protegez-vous.ca/sante-et-alimentation/supplements-proteines/le-boom-de-la-creatine-ce-que-vous-devrie
2. *nutritionniste-paris.com* — risques et contre-indications : https://www.nutritionniste-paris.com/2025/09/25/risque-creatine/
3. *topmuscu.fr* — synthese des etudes 2024–2025 : https://www.topmuscu.fr/dernieres-etudes-sur-la-E2%80%912025/
4. *passeportsante.net* — fiche complement alimentaire : https://www.passeportsante.net/fr/Solutions/PlantesSupplements/Fiche.aspx?doc=creatine_ps
5. *National Geographic France* — effets cognitifs et cerebraux : https://www.nationalgeographic.fr/sciences/2025/02/sante-performance-anxiete-sommeil-la-creatine-un-complement-alimenta

1.2. Pipeline de crawling

Pour extraire le texte utile des pages, on a utilise `trafilatura` qui isole automatiquement le contenu editorial (suppression des menus, pubs, pieds de page). Avant chaque requete, on verifie le fichier `robots.txt` du site via `urllib.robotparser` pour respecter les regles d'accès. On rejette aussi les pages avec moins de 500 mots : en dessous de ce seuil, il s'agit généralement d'une page de redirection ou d'un contenu trop pauvre pour en tirer des entités utiles. Les textes valides sont sauvegardes ligne par ligne dans un fichier JSONL (`crawler_output.jsonl`) avec l'URL, le nombre de mots et le contenu.

On a aussi ajouté un delai d'une seconde entre chaque requête (`time.sleep(1)`) pour ne pas surcharger les serveurs, et un timeout de 20 secondes pour les pages lentes.

1.3. Extraction d'entités nommées (NER)

La NER est realisée avec spaCy et le modèle `fr_core_news_sm`. On garde quatre categories : `PER` (personnes), `ORG` (organisations), `LOC` (lieux) et `DATE`. Un filtre supplémentaire (`is_valid_entity`) élimine les faux positifs évidents : entités de moins de 3 caracteres, tokens dont le mot principal est un verbe, adjectif ou determinant, et noms contenant des apostrophes.

Pour les triplets RDF, on parcourt chaque phrase : si elle contient au moins deux entités valides et un verbe, on génère un triplet (entite1, lemme_verbe, entite2).

Table 1: Exemples d'entites extraites sur le corpus creatine

Entite	Type	Contexte
Scott Forbes	PER	chercheur cite dans plusieurs articles
Universite de Brandon	ORG	affiliation du chercheur
Manitoba	LOC	province canadienne
2024	DATE	annee de publication d'etudes
National Geographic	ORG	source journalistique

1.3.1. Trois cas d'ambiguite

1) ATP classe comme ORG. Le modèle français interprète l'acronyme ATP comme une Organisation (il pense probablement au circuit de tennis). Dans notre corpus, ATP désigne l'Adenosine Triphosphate, une molécule. Sans contexte biochimique, spaCy ne peut pas faire la

distinction.

2) Termes scientifiques non détectés. La créatinine, la phosphocreatine, l'ATP en tant que molecule, ou encore le collagène ne sont pas reconnus comme entités nommées par `fr_core_news_sm` car ce sont des noms communs scientifiques, pas des noms propres. On a compensé en ajoutant Manuellement 45 triplets du domaine directement dans la base (voir section 2).

3) Bruit lexical. Certains tokens avec une majuscule en début de phrase ("Combien", "Envie") sont capturées comme entités. On les a filtrés avec la fonction `is_valid_entity` qui vérifie que le token principal est bien un nom.

2. Construction de la KB, alignement et expansion

2.1. Graphe initial (KB privée)

Les triplets RDF sont construits à partir des phrases avec deux entités et un verbe. Chaque composant est converti en URI :

- Entités : `http://example.org/resource/{label}`
- Prédicats : `http://example.org/property/{lemme}`
- Labels : `rdfs:label` en francais pour la lisibilite

Le NER seul ne donnait pas assez de triplets (le modele rate tous les termes scientifiques). On a donc ajouté manuellement 45 triplets couvrant le domaine : relations entre la créatine et ses effets (`hasEffect`, `storedIn`, `improves`), les chercheurs et leurs affiliations, les molécules de la chaîne énergétique (phosphocreatine, ATP), et les maladies étudiées. La KB initiale atteint ainsi plus de 100 triplets et 50 entités, conforme aux exigences du TP.

2.2. Alignement d'entites avec Wikidata

Pour chaque entite de notre KB privée, on interroge l'API `wbsearchentities` de Wikidata avec le nom en anglais. Le premier résultat est retenu avec un score de confiance issu de la qualité du match. Quand une entité est trouvée, on ajoute un lien `owl:sameAs` vers son URI Wikidata. Quand elle n'existe pas dans Wikidata, on la definit localement comme membre de la classe `ex:CreatineRelatedConcept`.

On a aussi défini une mini-ontologie avec `OWLReady2` : hiérarchie de classes (`CreatineSupplement` $\subset$ `DietarySupplement` $\subset$ `Substance`), domaines et rangs des propriétés principales (`hasEffect`, `affiliatedWith`, `studiedBy`).

Les entités core alignées vers Wikidata : creatine (Q223600), phosphocreatine (Q1984607), ATP (Q80863), muscle (Q7365), brain (Q897386), kidney (Q9377), depression (Q42844), anxiety (Q154430), Alzheimer (Q11081), Natural Killer cells (Q332181).

2.3. Alignement des predicats

Les prédicats locaux sont alignés manuellement vers des propriétés Wikidata via deux méthodes :

- **Methode 1** : requêtes SPARQL directement entre deux entités alignées pour trouver les propriétés existantes
- **Methode 2** : recherche par mot-clé dans les labels des propriétés Wikidata.

2.4. Expansion de la KB via SPARQL

L'objectif est d'atteindre 50 000 triplets. On est parti des 11 QIDs alignés, mais l'expansion 1-hop n'a donne que quelques centaines de triplets insuffisant. L'expansion 2-hop entité par entité était trop lente (timeouts frequents). La solution adoptée : des requêtes **predicate-controlled** qui récupèrent tous les triplets d'une classe entière depuis Wikidata en une seule requete. Quatre requêtes ont été lancées :

Table 2: Decisions d'alignement des predicats

Predicat local	Wikidata	Relation	Methode
`locatedIn`	P131	owl:equivalentProperty	M1 (Manitoba–Canada)
`affiliatedWith`	P1416	owl:equivalentProperty	M1 (Forbes–Brandon)
`publishedIn`	P1433	owl:equivalentProperty	M1 (Forbes–Sci. Reports)
`hasEffect`	P1542	owl:equivalentProperty	M2 (mot-cle "effect")
`isA`	P31	owl:equivalentProperty	M2 (mot-cle "instance")
`storedIn`	P276	rdfs:subPropertyOf	M2 (plus general)
`studiedBy`	P1450	owl:equivalentProperty	M2 (mot-cle "researcher")

- Composés chimiques (`wdt:P31 wd:Q11173`) – 20 000 triplets
- Suppléments alimentaires (`wdt:P31 wd:Q188055`)
- Proteines (`wdt:P31 wd:Q8054`) – 20 000 triplets
- Sous-classes de composés chimiques (`wdt:P279 wd:Q11173`)

Apres nettoyage (suppression des préfixes schema.org, wikibase, mediawiki) et filtrage aux **150 relations les plus fréquentes**, la KB finale respecte toutes les contraintes :

Table 3: Statistiques finales de `expanded_kb.nt`

Dimension	Valeur	Cible	Statut
Triplets	67 234	50k – 200k	OK
Entites	∼20 927	5k – 30k	OK
Relations	150	50 – 200	OK

3. Raisonnement SWRL

3.1. Règle sur family.owl

On a charge l'ontologie `family.owl` avec `OWLReady2` et applique la regle suivante pour inférer la classe `oldPerson` :

$$\texttt{Person}(p) \ \wedge \ \texttt{age}(p,\, a) \ \wedge \ a > 60 \implies \texttt{oldPerson}(p)$$

La classe `oldPerson` a été créée comme sous-classe de `Person` avant de définir la règle. Le raisonneur Pellet (`sync_reasoner_pellet`) a correctement inféré tous les individus dont l'âge dépasse 60 ans, validé par verification manuelle sur l'ensemble des individus de l'ontologie.

3.2. Règle sur la KB creatine

Sur notre propre KB, on a défini une regle Horn exprimant : *si une entité est une instance d'humain et a pour occupation le metier de scientifique, alors elle a probablement reçu un prix* :

$$\texttt{p31}(x,\ \text{humain}) \ \wedge \ \texttt{p106}(x,\ \text{scientifique}) \implies \texttt{p166}(x,\ \text{prix})$$

Cette règle a aussi servi a tester empiriquement la composition vectorielle dans la partie KGE (section suivante) : on vérifie si $\vec{p31} + \vec{p106} \approx \vec{p166}$ dans l'espace d'embedding.

4. Knowledge Graph Embeddings (KGE)

4.1. Préparation des données

Les triplets de `expanded_kb.nt` sont filtrés pour ne garder que les triplets URI–URI–URI (sans littéraux), puis normalisés (minuscules, underscores). Le split 80/10/10 est effectué avec `random.seed(42)` pour la reproductibilité. Les entités "orphelines" presentes dans valid/test mais absentes du train sont transférées vers le train pour éviter l'évaluation sur des entités inconnues.

Table 4: Splits après correction des entités orphelines

Ensemble	Proportion	Taille
Train	80%	∼52 681 triplets
Validation	10%	∼4 786
Test	10%	∼4 777

4.2. Modèles et résultats

Deux modèles ont été entraînés avec PyKEEN (100 epoques, Adam lr=0.001, 10 negatifs par positif, embedding dim=100, GPU) :

TransE modèle les relations comme des translations vectorielles : $\vec{e}_s + \vec{r}_p \approx \vec{e}_o$.

DistMult utilise un produit élémentaire bilineaire : $\text{score}(s, p, o) = \langle \vec{e}_s, \vec{r}_p, \vec{e}_o \rangle$. Par construction il est symetrique, ce qui est un avantage pour les relations comme `sameAs` tres présentes dans Wikidata.

Table 5: Métriques de link prediction (both, realistic)

Modèle	MRR	Hits@1	Hits@3	Hits@10
TransE	0.075	0.009	0.104	0.182
DistMult	0.266	0.179	0.316	0.421

DistMult domine TransE sur toutes les métriques. C'est attendu : Wikidata contient beaucoup de relations symétriques (N-to-N, `sameAs`, `spouseOf`) que TransE géré mal car son hypothèse de translation ne tient pas pour ces patterns. TransE obtient cependant un Hits@10 a 22.7% : la bonne entité est dans le top 10 dans presque un quart des cas, ce qui est correct pour un graphe de 20 000 entités.

4.3. Sensibilité a la taille du KB

DistMult a été entraîné sur trois sous-ensembles pour mesurer l'impact de la taille des données :

Table 6: Impact de la taille du KB sur DistMult

Taille train	MRR	Conclusion
20k triplets	0.100	Performance de base
50k (= full)	0.212	MRR double

Doubler la taille du graphe double le MRR. Ça montre que pour les KGE, la quantité de données est souvent plus déterminante que le choix du modèle. On n'a pas pu tester un palier 50k distinct car notre KB fait exactement ∼46k triplets après nettoyage, donc "50k" et "full" sont équivalents.

4.4. Analyse des embeddings

Plus proches voisins. La similarité cosinus entre vecteurs d'entités de DistMult montre que les entités proches dans l'espace vectoriel partagent le meme voisinage relationnel dans le graphe. Les clusters ne correspondent pas aux plages d'ID Wikidata mais à des groupes d'entités ayant les mêmes types de relations — l'embedding capture la structure du graphe, pas la popularite.

t-SNE. 2 000 entités projetées en 2D (perplexite 30, 1000 iterations) : même observation que ci-dessus, pas de clustering par popularite (ID Wikidata), mais par voisinage relationnel.

Comportement des relations. Les normes des vecteurs de relations révèlent les prédicats les plus "actifs" dans l'espace. Les relations `p31` (instance of) et `p279` (subclass of) ont une forte similarité cosinus, ce qui est cohérent : elles partagent une sémantique proche de classification.

Composition regle Horn. Le test $\vec{p31} + \vec{p106} \approx \vec{p166}$ donne un rang de 40/200 (similarité 0.10). DistMult ne capture pas cette composition. C'est une limite connue : le produit élémentaire

n'encode pas les chemins relationnels. TransE ou RotatE auraient mieux supporte ce type de composition.

5. RAG sur RDF/SPARQL

5.1. Setup

- Machine : Windows local, CPU uniquement
- Modele LLM : `llama3.2:1b` via Ollama (alternative a gemma:2b qui refusait les taches techniques)
- Graphe source : `expanded_kb.nt` — 67 234 triplets

5.2. Schema summary

Pour que le LLM puisse générer du SPARQL valide, on lui fournit un resume du schema du graphe injecte dans chaque prompt : les prefixes (namespaces utilises), jusqu'a 80 prédicats distincts, 40 classes distinctes, et 20 triplets échantillons. Ce contexte permet au modele de connaitre la structure du graphe sans charger tous les triplets.

5.3. Pipeline NL–SPARQL avec self-repair

Le pipeline suit trois étapes :

1. **Génération** : le LLM reçoit le schema summary + la question et génère une requête SPARQL dans un bloc de code. Le prompt precise de n'utiliser que les IRIs du schema.
2. **Execution** : la requête est exécutée sur le graphe via rdflib. En cas d'erreur de syntaxe, on passe à l'étape suivante.
3. **Self-repair** : l'erreur + la mauvaise requête sont renvoyées au LLM pour correction. La requête réparée est re-executée.

5.4. Evaluation : baseline vs RAG

On a évalué le pipeline sur 5 questions représentatives du domaine :

Table 7: Comparaison baseline vs RAG sur 5 questions

Question	Baseline	RAG	Repare ?
Effects of creatine?	Réponse générale correcte	0 résultats	oui
Instances of chemical compound?	Réponse générale correcte	0 résultats	oui
What is creatine instance of?	Réponse générale correcte	0 résultats	non
Entities related to Alzheimer?	Reponse generale correcte	0 résultats	oui
Properties of Q223600?	Ne connait pas Q223600	0 résultats	oui

Analyse des résultats. Le baseline (LLM sans KB) répond correctement aux 4 premieres questions car la creatine est un sujet populaire dans les données d'entrainement du modele. La question 5 (`Q223600`) illustre exactement le cas d'usage du RAG : le LLM sans contexte ne sait pas ce qu'est cet identifiant Wikidata, alors que le graphe contient toutes ses propriétés. Malheureusement, la génération SPARQL est insuffisante pour en profiter.

Cas d'echec. Le modèle `llama3.2:1b` (1 milliard de parametres) utilise systematiquement des prefixes inventes (`brick:`, `skos:`, `dcat:`) absents du graphe. La boucle de self-repair corrige les erreurs de syntaxe SPARQL dans 4 cas sur 5, mais ne peut pas corriger les hallucinations d'IRIs — le modèle ne "sait" pas quelles URIs existent vraiment dans notre graphe.

Interface CLI. Une fonction `run_cli_demo()` a été implementée pour la demo interactive. Elle affiche pour chaque question : la réponse baseline, la requête SPARQL générée, les résultats du graphe ou l'erreur + la tentative de réparation. Elle est lancée en ligne de commande via `python lab6_rag.py` (commentée dans le notebook car `input()` ne fonctionne pas bien dans Jupyter).

6. Réflexion critique

NER : le maillon faible. Le modèle `fr_core_news_sm` est trop généraliste pour du vocabulaire scientifique. Il rate toutes les molécules (creatine, ATP, phosphocreatine) car ce sont des noms communs, pas des noms propres. On a compensé avec les triplets manuels, mais dans un vrai projet on utiliserait un modèle NER fine-tune sur des textes biomedicaux (CamemBERT-bio, par exemple).

Expansion : compromis quantite vs coherence. L'expansion predicate-controlled sur des classes Wikidata génériques ("tous les composés chimiques") a permis d'atteindre 67 000 triplets, mais au prix d'un bruit important : beaucoup d'entités n'ont aucun rapport avec la creatine. C'est visible dans les nearest neighbors ou certains voisins sont surprenants. Une expansion plus ciblée sur des sous-classes biochimiques aurait donné un graphe plus cohérent.

DistMult vs TransE. DistMult performe mieux ici car Wikidata est plein de relations symetriques (sameAs, spouseOf, N-to-N). Mais DistMult échoue sur la composition de relations (rang 40/200 pour la regle Horn). TransE aurait mieux supporte ce type de raisonnement, meme s'il performe moins bien globalement. C'est le compromis classique entre symetrie et compositionnalite.

CWA vs OWA. Les modeles KGE supposent le monde fermé (un triplet absent = faux), alors que Wikidata fonctionne en monde ouvert (absent = inconnu). Les négatifs générés par corruption peuvent en réalité être vrais, ce qui dégrade le signal d'apprentissage. C'est une limitation fondamentale difficile à contourner sans données négatives annotées.

RAG : limite du petit modèle. Le pipeline RAG est fonctionnel (schema summary, génération SPARQL, self-repair, comparaison baseline vs RAG), mais le modèle 1B ne fait pas une génération SPARQL fiable. La solution naturelle serait un modèle 7B+ (Mistral, Llama3-8B) ou un modèle fine-tune sur la génération SPARQL. L'impact du lab 4 est aussi a noter : un graphe avec des labels `rdfs:label` en anglais donnant des URIs lisibles ameliorerait significativement la génération SPARQL.

Ce qu'on ameliorerait.

- (1) Modele NER biomedical francais pour mieux capturer les entites du domaine
- (2) Ontologie OWL explicite avec hierarchie de classes exploitable par le raisonneur
- (3) RotatE ou ComplEx pour mieux gerer les relations asymetriques et la composition
- (4) Modele 7B+ pour la generation SPARQL dans le pipeline RAG.